PIZZAHUT

SQL SALES ANALYSIS

A business-focused analysis of pizza orders, revenue, product mix, and ordering patterns using MySQL.

PROJECT REPORT
Prepared for a GitHub Data Analyst portfolio

SQL • MySQL • Aggregations • Joins • Window Functions

Purpose

This report documents the questions addressed by the SQL queries, explains the analytical approach, highlights important interpretation details, and recommends improvements before publishing the project. It focuses on what the queries can tell a business—not just how the SQL works.

Important: no query result values or dataset outputs were provided with the SQL. Therefore, this report explains the analysis and expected interpretation without inventing sales figures, winning products, or business findings.

1. Executive Summary

The PizzaHut project uses relational SQL analysis to examine sales performance and customer ordering patterns. The queries combine order records with order-line, pizza, and pizza-type data to answer questions about order volume, sales revenue, product popularity, time-of-day demand, category mix, daily demand, and revenue contribution.

Analysis area	Business question	SQL approach
Order volume	How many orders were placed?	COUNT() on orders
Revenue	How much sales value was generated?	SUM(quantity × price)
Product pricing	Which pizza has the highest listed price?	JOIN + ORDER BY + LIMIT
Demand / popularity	Which sizes and pizza types are ordered most?	GROUP BY + COUNT/SUM
Demand timing	At which hours are orders placed?	HOUR() + GROUP BY
Daily demand	What is the average quantity sold per date?	Daily aggregation + AVG()
Revenue mix	Which categories/types contribute the most revenue?	Revenue aggregation + percentage calculation
Within-category ranking	What are the top three pizza types by revenue in each category?	RANK() window function

Overall assessment

The query set demonstrates foundational and intermediate SQL skills: joins across multiple tables, grouping and aggregation, sorting and limiting results, scalar subqueries, date/time extraction, and ranking with a window function. A few queries would benefit from clearer aliases, more precise metric definitions, and corrected grouping logic.

The most important analytical caution is to distinguish order count, pizza quantity, and revenue. These are different measures and should not be treated as interchangeable.

2. Data Model and Analytical Flow

The queries imply a four-table relational structure:

Table	Role in the analysis	Key relationship used
orders	Order-level information such as order ID, date, and time	orders.order_id
orders_details	Line-level records, including pizza ID and quantity	orders_details.order_id; pizza_id
pizzas	Pizza variants, size, and listed price	pizza_id; pizza_type_id
pizza_types	Pizza names and categories	pizza_type_id

Join path for product-level sales: `pizza_types` → `pizzas` → `orders_details`. To analyze order dates or times together with line quantities, join `orders` → `orders_details`.

Assumption to verify: the report assumes each row in `orders_details` represents an order line and that `pizzas.price` is the applicable unit price. If prices vary over time or discounts/taxes are stored elsewhere, the calculated revenue may differ from actual recognized revenue.

3. Query-by-Query Analysis

3.1 Total number of orders

```
SELECT COUNT(order_id) AS total_orders FROM orders;
```

What it does: Counts non-NULL order IDs in the orders table. If order_id is a primary key and never NULL, this equals the number of order records.

Business interpretation: Use this as an order-volume KPI. It does not measure the number of pizzas sold or the number of customers.

Suggested improvement: COUNT(*) is a common alternative when every row should be counted.

3.2 Total revenue from pizza sales

```
SELECT ROUND(SUM(od.quantity * p.price), 2) AS total_sales
FROM orders_details od
JOIN pizzas p ON p.pizza_id = od.pizza_id;
```

What it does: Multiplies each order line's quantity by the listed price, sums the values, and rounds the final result to two decimal places.

Business interpretation: This estimates gross product sales using the price stored in pizzas. It is not necessarily net revenue if discounts, refunds, taxes, or price history are not represented.

Suggested improvement: Use clear table aliases and name the metric total_sales or estimated_gross_sales.

3.3 Highest-priced pizza

```
SELECT pt.name, p.price
FROM pizza_types pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
ORDER BY p.price DESC
LIMIT 1;
```

What it does: Sorts pizza variants by price from highest to lowest and returns one row.

Business interpretation: This finds the highest-priced variant, not necessarily the most profitable or best-selling pizza. If several variants tie for the highest price, LIMIT 1 returns only one of them.

Suggested improvement: To show all ties, compare price with a MAX(price) subquery or use RANK(). Include size if variants of the same type have different prices.

3.4 Most common pizza size ordered

```
SELECT p.size, COUNT(od.order_details_id)
FROM pizzas p
JOIN orders_details od ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY p.size;
```

What it does: Counts order-detail rows for each size, then sorts the size labels alphabetically/numerically as stored.

Business interpretation: Important correction: this counts line items, not the number of pizzas. A line with quantity 5 counts once. It also does not rank sizes by popularity because it sorts by size.

Suggested improvement: For units sold, use SUM(od.quantity) and ORDER BY units_sold DESC. For order-line frequency, keep COUNT() but sort by the count DESC.

3.5 Top five pizza types by quantity

```
SELECT pt.name, SUM(od.quantity) AS quantity
FROM pizza_types pt
JOIN pizzas p ON pt.pizza_type_id = p.pizza_type_id
JOIN orders_details od ON od.pizza_id = p.pizza_id
GROUP BY pt.name
ORDER BY quantity DESC
LIMIT 5;
```

What it does: Adds quantities across all size variants for each pizza type, then returns the five largest totals.

Business interpretation: This is a strong product-demand query, assuming each pizza type name uniquely identifies the type. It measures units sold, rather than the number of order lines.

Suggested improvement: Consider grouping by both pt.pizza_type_id and pt.name to avoid merging different types that happen to share a name.

3.6 Distribution of orders by hour

```
SELECT HOUR(order_time) AS order_hour, COUNT(order_id) AS order_count
FROM orders
GROUP BY HOUR(order_time)
ORDER BY order_hour;
```

What it does: Extracts the hour component from order_time and counts orders in each hour.

Business interpretation: This reveals when orders are placed and can support staffing or campaign timing. The original query has no ORDER BY, so result order is not guaranteed.

Suggested improvement: Alias the hour and count fields, and sort by hour. Hours with zero orders will not appear unless you join to a complete hour dimension.

3.7 Category-wise distribution of pizzas

```
SELECT category, COUNT(name) AS pizza_types_count
FROM pizza_types
GROUP BY category;
```

What it does: Counts pizza-type records in each category.

Business interpretation: This describes the number of listed pizza types per category—not the distribution of pizzas actually sold. The comment mentions joining tables, but the query uses only pizza_types.

Suggested improvement: If the goal is sales distribution, join to pizzas and orders_details and aggregate SUM(quantity) by category. If the goal is catalog mix, the current approach is suitable.

3.8 Average pizzas ordered per day

```
SELECT AVG(daily_quantity) AS avg_pizzas_per_day
FROM (
SELECT o.order_date, SUM(od.quantity) AS daily_quantity
FROM orders o
JOIN orders_details od ON o.order_id = od.order_id
GROUP BY o.order_date
) AS daily_orders;
```

What it does: First calculates total pizza units for each date, then averages those daily totals.

Business interpretation: The aggregation pattern is correct for average units per date with at least one order. Dates with no orders are excluded, so this is not an average across every calendar day in a date range.

Suggested improvement: Use a calendar table if zero-order days must be included. Also note that the average of daily totals differs from total units divided by number of orders.

3.9 Top three pizza types by revenue

```
SELECT pt.name, SUM(od.quantity * p.price) AS revenue
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id = pt.pizza_type_id
JOIN orders_details od ON od.pizza_id = p.pizza_id
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 3;
```

What it does: Calculates sales value for each pizza type and returns the three largest revenue totals.

Business interpretation: This identifies revenue leaders, which may differ from quantity leaders because pizza sizes and prices differ.

Suggested improvement: Group by pizza_type_id and name; round revenue for display if desired. The metric is estimated sales revenue based on listed prices.

3.10 Percentage revenue contribution by category

```
SELECT pt.category,
ROUND(SUM(od.quantity * p.price) /
(SELECT SUM(od2.quantity * p2.price)
FROM orders_details od2
JOIN pizzas p2 ON p2.pizza_id = od2.pizza_id) * 100, 2)
AS revenue_percentage
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id = pt.pizza_type_id
JOIN orders_details od ON od.pizza_id = p.pizza_id
GROUP BY pt.category
ORDER BY revenue_percentage DESC;
```

What it does: Calculates revenue by category and divides it by total sales across all order lines to get a percentage.

Business interpretation: The intended calculation is appropriate. The original alias 'revenue' is misleading because the result is a percentage, not a revenue amount. The denominator can be simplified and should use the same revenue definition as the numerator.

Suggested improvement: Name the output revenue_percentage. Validate that category percentages sum to approximately 100%, allowing for rounding.

3.11 Top three revenue-generating types within each category

```
SELECT category, name, revenue
FROM (
SELECT category, name, revenue,
RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
FROM (
SELECT pt.category, pt.name,
SUM(od.quantity * p.price) AS revenue
FROM pizza_types pt
JOIN pizzas p ON p.pizza_type_id = pt.pizza_type_id
JOIN orders_details od ON od.pizza_id = p.pizza_id
GROUP BY pt.category, pt.name
) AS sales_by_type
) AS ranked_types
WHERE rn <= 3;
```

What it does: Aggregates revenue by category and pizza type, ranks each category independently, then returns ranks one through three.

Business interpretation: This is a useful window-function example. RANK() preserves ties, so a category can return more than three rows if multiple types share a rank at the cutoff. The original outer SELECT omits category, making the output less informative.

Suggested improvement: Include category in the final SELECT. Use ROW_NUMBER() if exactly three rows per category are required; use DENSE_RANK() if ranks should not skip after ties.

---

4. Technical Review: What to Fix or Clarify

Priority	Issue	Why it matters	Recommended action
High	Pizza size query counts rows and sorts by size	It does not answer which size sold the most units.	Use SUM(quantity) and sort DESC for units sold.
High	Category query counts catalog names only	Catalog availability is different from customer demand.	Clarify whether the goal is catalog mix or sold quantity/revenue.
High	Revenue is based on current/listed pizza price	It may not equal actual historical revenue.	Document the assumption; include discounts/taxes only if available.
Medium	Revenue percentage alias says 'revenue'	Readers may mistake a percentage for a currency value.	Rename it to revenue_percentage.
Medium	Top-three-per-category output omits category	Readers cannot see which category each result belongs to.	Select category in the final query.
Medium	Some GROUP BY clauses use name only	Duplicate names could be combined into one group.	Group by stable ID and display name.
Low	Hourly query has no explicit ordering	Output order is not guaranteed.	Add ORDER BY order_hour.
Low	Daily average excludes days without orders	The denominator is active sales dates only.	Use a calendar table if all calendar days are in scope.

Suggested corrected query: most popular pizza size by units sold

```
SELECT p.size,
SUM(od.quantity) AS units_sold
FROM pizzas AS p
JOIN orders_details AS od
ON od.pizza_id = p.pizza_id
GROUP BY p.size
ORDER BY units_sold DESC
LIMIT 1;
```

Suggested corrected query: category-level sales distribution

```
SELECT pt.category,
SUM(od.quantity) AS units_sold,
ROUND(SUM(od.quantity * p.price), 2) AS estimated_revenue
FROM pizza_types AS pt
JOIN pizzas AS p
ON p.pizza_type_id = pt.pizza_type_id
JOIN orders_details AS od
ON od.pizza_id = p.pizza_id
GROUP BY pt.category
ORDER BY estimated_revenue DESC;
```

5. Business Questions This Project Can Answer

Business area	How to use the analysis	Potential decision
Sales performance	Track order count and estimated sales value.	Compare periods after adding date filters or monthly trends.
Product strategy	Compare top types by units sold versus revenue.	Promote high-demand products and review pricing/mix for revenue leaders.
Menu planning	Compare sales by size and category.	Adjust inventory and menu emphasis based on actual demand.
Operations	Review orders by hour.	Align staffing and preparation capacity with peak order periods.

Business area	How to use the analysis	Potential decision
Demand planning	Use average daily pizza quantity.	Create a baseline for purchasing and production planning.
Category strategy	Measure each category's share of sales.	Assess whether revenue depends heavily on one category.

These are potential uses of the outputs, not confirmed findings. To make evidence-based recommendations, run the queries, record the actual results, and interpret them alongside the date range and data limitations.

6. SQL Skills Demonstrated

Skill	Where it appears	Why it matters
Aggregate functions	COUNT, SUM, AVG, ROUND	Summarize orders, units, and estimated sales.
Joins	orders_details ↔ pizzas ↔ pizza_types; orders ↔ orders_details	Combine normalized tables for analysis.
Grouping and sorting	GROUP BY, ORDER BY, LIMIT	Compare categories and rank leading products.
Subqueries	Daily-average and total-revenue denominator	Build calculations from intermediate or overall totals.
Date/time functions	HOUR(order_time)	Identify time-of-day order patterns.
Window functions	RANK() OVER (PARTITION BY category ...)	Rank products independently within each category.

7. Recommended GitHub Project Structure

```
PizzaHut-SQL-Analysis/
■■■ README.md
■■■ data/
■ ■■■ (dataset files or source link)
■■■ sql/
■ ■■■ 01_basic_sales_metrics.sql
■ ■■■ 02_product_and_category_analysis.sql
■ ■■■ 03_time_and_daily_trends.sql
■ ■■■ 04_revenue_ranking.sql
■■■ reports/
■■■ PizzaHut_SQL_Sales_Analysis_Report.pdf
```

In the README, include the project objective, dataset source and date range, schema/table descriptions, SQL questions, key results with actual numbers, limitations, and screenshots or charts if available. Keep the raw SQL scripts reproducible and add comments that explain the business question behind each query.

8. Conclusion

This PizzaHut SQL project is a solid beginner-to-intermediate sales analytics case study. It shows how SQL can convert relational order data into measures of order volume, product demand, estimated revenue, category contribution, and time-of-day patterns. Its strongest technical elements are the multi-table joins, daily aggregation, revenue calculations, and category-partitioned ranking.

Before publishing, prioritize fixing the pizza-size popularity query, clarifying the category-distribution metric, renaming the revenue-percentage output, and including category in the final ranking result. Then run the final queries and add real findings to the README. This will make the project more credible and useful to recruiters reviewing a Data Analyst portfolio.

Metric definitions to keep consistent

Metric	Definition used in this report
Orders	Count of records/order IDs in orders.
Units sold	Sum of orders_details.quantity.
Order-line count	Count of rows/order_details IDs in orders_details.
Estimated revenue	Sum of quantity × listed pizza price.
Revenue contribution	Category/type estimated revenue ÷ total estimated revenue × 100.

Prepared from the SQL statements supplied for this project. Results are not included because execution outputs were not supplied.